

DecryptChain: A Permissionless Proof-of-Work Encrypted Mempool

Nicolas Alhaddad
Boston University, USA
nhaddad@bu.edu

Alireza Kavousi
University College London, UK
a.kavousi@cs.ucl.ac.uk

Abstract

Blockchain mempool transparency fuels Maximal Extractable Value (MEV), where attackers can front-run, back-run, and reorder transactions as soon as they appear. Encrypted mempools aim to delay the release of information until block commitment, yet nearly all existing designs rely on a trusted decryption committee. This creates two structural problems. First, committee members hold decryption material by design, so a colluding threshold can reconstruct the decryption key and learn transactions before block commitment. Second, once such a committee becomes malicious, honest parties have no easy in-protocol way to recover: restoring privacy for future epochs requires an external intervention such as a hard fork that replaces the committee and rotates the long-lived cryptographic material.

In this work, we ask whether encrypted mempools can instead use proof-of-work to realize an open and recoverable decryption committee. We then introduce DecryptChain, a permissionless proof-of-work encrypted mempool in which decryption authority is not assigned to persistent identities or long-lived key shares. Instead, decryption is continuously re-contested through public computational work. Even if an adversary successfully breaches one epoch, it gains no reusable secret for future epochs; honest parties can always re-enter and recover the decryption process by contributing sufficient work. DecryptChain decouples block production from decryption, enabling it to operate as a Layer-2 timely decryption service on any underlying blockchain while preserving eventual decryption for committed on-chain encrypted transactions.

1 Introduction

Blockchain transparency creates a fundamental tension for transaction privacy. Pending transactions are visible before their final order is fixed, which allows sophisticated parties to exploit their contents. This problem is especially visible in Maximal Extractable Value (MEV) [11]. Estimates put the cumulative MEV on Ethereum in the billions of dollars

since 2020,¹ underscoring the need for privacy at the mempool layer. An *encrypted mempool* addresses this problem by keeping pending transactions encrypted until the relevant ordering decision has been fixed. The goal is not to hide that a transaction exists, but to prevent parties who can influence or observe ordering from learning transaction contents early. The common method to implement an encrypted mempool is a commit-reveal paradigm [21], where encrypted transactions are first committed, and only after commitment (*i.e.*, fixed ordering) are opened/decrypted.

The systems realizing encrypted mempools typically contain a *pre-existing shortcut to opening*: some privileged secret, setup state, hardware capability, or any access structure that exists before transaction commitment, and lets its holder recover the opening information faster than the intended public process. The fact that such a shortcut, privileged opening capability already exists before ordering is fixed, turns out to be problematic in the context of encrypted mempools involving rational actors who might misuse this information to gain MEV. Existing encrypted mempool designs differ mainly in how they try to keep opening information unavailable during the commit phase. Threshold-cryptographic systems assign opening material to a designated set of parties, as in Ferveo [3], BlindPerm [22], and recent batched threshold encrypted mempools [9]. TEE-based systems place the opening capability inside hardware isolation [27], thereby relying on hardware assumptions and inheriting the fragility of TEE threat models [35]. Timed-release approaches instead try to make opening depend on delayed computation, typically through time-lock puzzles or related primitives [12, 25, 33].

These approaches make different efficiency and trust trade-offs. Threshold and TEE systems can provide efficient batch opening, but they also create privileged opening capability before commitment. Timed-release systems reduce reliance on ordinary validator committees, but timing assumptions are difficult to calibrate against specialized hardware, and delayed computation for every transaction or small batch can

¹<https://blog.shutter.network/industry-leaders-propose-ethereums-first-threshold-encrypted-mempool/>

harm scalability. Moreover, timed-release instantiations with trusted-setup [7] may themselves introduce opening shortcuts, as whoever knows the relevant setup secret can compute the delayed output faster than the prescribed public process. Time-lock encryption [18] gives efficient timed release, but typically reintroduces a designated service or committee that participates in producing timed decryption material.

In this paper, we isolate the early recovery of opening information as the primary cryptographic attack vector in commit-reveal encrypted mempools. If an adversary obtains the opening trapdoor before the ordering decision is fixed, they can learn transaction contents prematurely and extract MEV. We abstract away orthogonal leakage channels such as metadata, timing, and side-channel leakage.² To formally reason about this timeline, we define an *epoch* as the operational lifespan of the active encryption instance (*i.e.*, the intended time interval required to recover the opening information). Also, the *commit window* represents the (shorter) interval during which the ordering decision for those ciphertexts is finalized. An *early opening* (or premature decryption) occurs if any party successfully recovers an epoch’s trapdoor before its corresponding commit window has closed.

Due to their practical efficiency and natural alignment with validator-based consensus, threshold-committee solutions have received significant attention in both practice [14] and academia [1, 10]. However, their reliance on pre-distributed decryption capabilities introduces two structural vulnerabilities. First, a committee compromise constitutes a *capability failure*, not merely an incentive failure. A colluding threshold, or a malicious setup authority, does not need to break the underlying cryptography; it simply exploits an opening capability that already exists prior to the commit phase. Furthermore, if future epoch keys are reshared or bootstrapped from this compromised state, an adversary can silently propagate this advantage across subsequent epochs, maintaining continuous premature access to the mempool until fresh, independent opening material is established.

Second, recovery from such a breach is neither automatic nor permissionless. Honest external actors cannot unilaterally repair the failure simply by deciding to participate (*e.g.*, by allocating computational power), as they lack the necessary secret shares or setup parameters required to intervene. Consequently, restoring system privacy necessitates out-of-band coordination to establish future opening information independently of the corrupted state. This typically requires executing an entirely new distributed key generation (DKG) setup [2], or in the worst case, executing a network hard-fork.³

Accountability mechanisms such as slashing, snitching, and audits can mitigate some forms of committee collusion [5, 13]. They are complementary to our goal, but they do not elimi-

nate the recovery issue. Detection may fail, attribution may be difficult, and the expected value of early access may exceed the expected penalty [24]. Even when collusion is detected and punished, future privacy is not restored unless the system replaces the compromised opening capability with fresh, independent opening material. This motivates the following research question:

Can we design an encrypted mempool that avoids reliance on pre-existing shortcuts with an epoch-local decryption that is open to participation and recoverable in later epochs?

We answer this question affirmatively with DecryptChain, a proof-of-work (PoW) encrypted mempool. The central idea is to replace pre-existing shortcuts to opening with an open computational race. No party receives a decryption share, setup trapdoor, TEE-resident opening capability, or any other privileged opening. Instead, each epoch has a public cryptographic puzzle, and any party can act as a decryptor by solving it. If an adversary solves one epoch early, it learns only that epoch’s opening information; it does not obtain any information that helps derive future epoch openings.

This structural vulnerability motivates our use of a proof-of-work (PoW) mechanism. The PoW layer is not a replacement for consensus, nor does it dictate transaction ordering. Rather, it ensures that the opening role is public, computationally costly, and independently re-earned in every epoch. A solver’s capability to open an epoch derives solely from expending work on that epoch’s active cryptographic instance, not from being selected as a validator, admitted to a committee, provisioned with a secret share, or trusted with hardware-protected state. Consequently, any adversarial advantage is computationally bounded, epoch-local, and contestable, rather than free, structural, and persistent. Honest parties can seamlessly enter subsequent epochs by contributing computational work, and a past decryption success confers no reusable advantage for future openings. Through this architecture, DecryptChain separates *ordering authority* from *opening authority*. The base chain remains responsible for ordering encrypted transactions and providing data availability. The PoW layer is responsible exclusively for opening (*i.e.*, decrypting) those ciphertexts after the relevant ordering decision has been finalized. This ensures that the underlying Layer-1 can retain its native consensus protocol undisturbed (*e.g.*, Proof-of-Stake BFT). DecryptChain effectively operates as a permissionless decryption service layered above the base chain; it does not entangle decryption work with the fork-choice rule or block-production mechanics.

This separation is critical because consensus failures and confidentiality failures exhibit different auditability profiles. When a consensus committee equivocates, censors, or stalls, the failure generates visible public evidence, such as conflicting blocks or delayed finality. By contrast, a confidentiality failure in an encrypted mempool is inherently silent: an adversary can prematurely recover plaintexts, extract MEV,

²While these leakage channels are critical in practice, they fall outside the scope of our formal cryptoeconomic bounds.

³Executing a new DKG may still not be sufficient to protect the privacy of ciphertexts under time-lock encryption for future epochs.

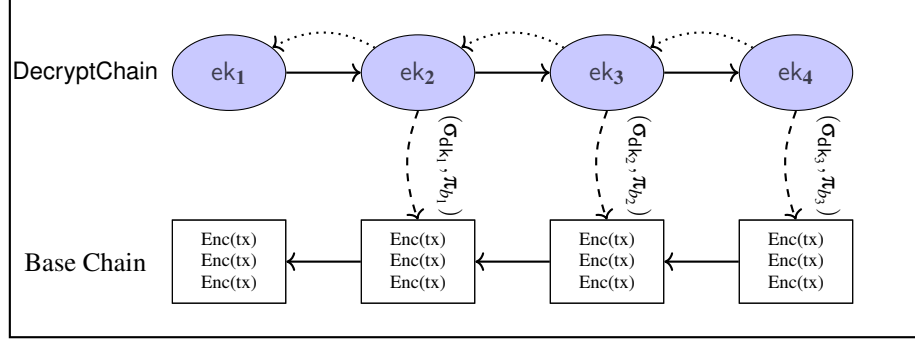


Figure 1: A high-level illustration of our DecryptChain architecture layered over a base chain. Time progresses in discrete epochs, where epoch i corresponds to the lifespan of the active encryption key ek_i . During the initial commit window of epoch i , users encrypt transactions under ek_i , and the base chain finalizes them into a committed batch b_i . The dashed arrows illustrate the cross-chain communication: upon successfully solving the active puzzle, the winning DecryptChain miner submits the updated state σ_{dk_i} and validity proof π_{b_i} to the base chain. As shown by the dotted arrows, the subsequent key ek_{i+1} extends ek_i and is deterministically generated on-chain only after the current epoch’s proof is verified.

and leave no public transcript proving who exploited the vulnerability. Therefore, while stake-weighted or permissioned committees are acceptable for ordering under standard assumptions, the opening process should not grant them pre-distributed decryption capabilities. DecryptChain resolves this by allowing the base chain to safely commit ordered ciphertexts, while an open, permissionless computational process enforces decryption post-commitment.

At a high level, the protocol operates in continuous epochs, each initialized with a unique public encryption key ek_i . Users encrypt their transactions under ek_i , and the base chain commits the resulting ciphertexts in a finalized order during the epoch’s commit window. The corresponding decryption key is subsequently recovered by an open decentralized committee of workers (or miners) who compete to solve the epoch’s active cryptographic puzzle. Upon success, the winning worker submits a verifiable, decrypted batch or (zero-knowledge) state update to the base chain. Crucially, the next epoch’s key (ek_{i+1}) is derived deterministically from the fresh Layer-1 state, ensuring that subsequent epochs act as strictly independent work instances rather than continuations of prior decryption authority. If decryption takes longer than the calibrated target time, the committed ciphertexts simply remain pending until their original puzzle is solved, and the observed latency is fed into a difficulty-adjustment mechanism to calibrate future epochs. Figure 1 gives a pictorial description of our protocol.

1.1 Overview of Our Results

Our main contributions are two-fold: we first establish a generalized formal taxonomy for evaluating encrypted mempools, and then present DecryptChain, a concrete system architecture that resolves the structural vulnerabilities identified by our model. While presented as a novel architectural blueprint, we

introduce several techniques to ensure the system satisfies the demands of large-scale deployment.

A formal model of premature decryption and recovery.

Section 3 models early opening as the event that a coalition obtains the relevant epoch-opening information before the transaction ordering decision has been fixed. This abstraction captures both systems with pre-existing shortcuts to opening (where decryption follows from threshold shares, setup state, or TEE-resident capabilities) and computation-based systems (where opening is obtained by solving a public work instance). The model separates two notions that are often conflated in the literature: preventing early opening in a single epoch and recovering from adversarial advantage in subsequent epochs. It classifies early-opening risk into three qualitative regimes: *Epoch-Easy*, where early opening is essentially available to a deviating coalition; *Epoch-Moderate*, where early opening has a non-negligible but bounded probability within the relevant commit window; and *Epoch-Hard*, where early opening is cryptographically negligible.

DecryptChain: A permissionless encrypted mempool.

Section 4 presents DecryptChain as a timely, permissionless decryption service layered over an existing base chain. By replacing pre-existing shortcuts with a continuous proof-of-work process, DecryptChain guarantees that adversarial advantage is strictly bounded, epoch-local, and automatically recoverable. To make this architecture practical, we develop the following core mechanisms:

- **Puzzle-based amortized decryption:** The construction instantiates each active cryptographic instance using a discrete-logarithm puzzle over a prime-order elliptic-curve group. Each public encryption key ($ek_i = g^{dk_i}$)

serves a dual purpose: it allows users to encrypt transactions, and it acts as the public puzzle whose solution is the matching trapdoor dk_i . One recovered epoch key opens a committed batch, achieving amortized decryption without assigning privileged opening authority.

- **Epoch-local key unpredictability:** To guarantee unpredictability, each epoch key is deterministically derived from finalized public Layer-1 state. As the public key is explicitly tied to epoch-specific public state, no party samples or receives the corresponding decryption key in advance. This prevents an adversary from choosing a favorable trapdoor, grinding over many keys, or precomputing solutions prior to the epoch’s commit window.
- **Theft-resistant state updates:** DecryptChain resolves the unique-solution theft problem created by permissionless decryption. To prevent front-running, the winning decryptor does not reveal dk_i directly on-chain. Instead, they post the executed state update together with a (zero-knowledge) validity proof (π_{b_i}) bound to the epoch, the committed ciphertext batch, and their payout address. We also detail a commit-reveal variant for universal decryption, in which revealing dk_i allows multiple standalone rollups to use the same decryption result.
- **Smooth difficulty adjustment:** We introduce a novel square-root difficulty adjustment mechanism that self-adjusts in a decentralized manner to provide *smooth* and *verifiable* control of target latency, which could be of independent interest. Instead of fixing a static security level, DecryptChain employs a feedback-driven mechanism to dynamically adjust the effective hardness of future discrete-logarithm puzzles based on observed decryption time. The base chain verifies the first valid proof and releases the reward (e.g., funded via fees) in the native asset to the bound payout address.

Table 1 compares DecryptChain with representative commit-reveal style encrypted mempool mechanisms. In the table, ① *Open* means that the opening role is not assigned to a designated decryptor or setup holder with a privileged shortcut. ② *BatchDec* means that decryption is amortized across a batch rather than requiring an interactive process or puzzle for every ciphertext. ③ *Recovery* means that an adversarial opening advantage in one epoch is temporary and can be overcome by honest entry in later epochs without an out-of-band procedure. ④ *NoSetup* means no trusted setup, trusted hardware, or pre-existing privileged shortcut to opening.

2 Preliminaries and Background

2.1 Basic Notations

Let $\mathbb{G}$ denote a cyclic group of prime order q , and $\mathbb{F}$ denote its scalar field. We denote the security parameters by λ for

Table 1: Comparison of commit-reveal style encrypted mempool mechanisms. A black circle states that the protocol satisfies a property, a white circle states that it does not, and a half circle states that the property depends on the instantiation or holds only under additional restrictions.

Protocol family	Open	BatchDec	Recovery	NoSetup
Committee-based [3, 9]	○	◐	○	○
TEE-based [27]	○	●	○	○
TLP/VDF-based [25, 33]	◐	○	◐	◐
CapsuleChain [12]	○	◐	◐	●
FairPoS [8]	◐	◐	◐	○
DecryptChain	●	●	●	●

computationally hard problems and by κ for computationally moderate problems. We use $x \leftarrow S$ to denote a uniform sampling from a finite set S . We use uppercase letters for group elements $G \in \mathbb{G}$, and lowercase letters for scalars $a \in \mathbb{F}$. The set $1, \dots, n$ is represented by $[n]$.

2.2 Encrypted L2

A Layer-1 (L1) blockchain is the base ledger that provides native consensus, data availability (DA), and state-transition enforcement under its own security model. It orders transactions and finalizes the canonical chain via its consensus protocol (e.g., PoW or PoS), while executing transactions according to its native rules. Other systems (e.g., L2s and sidechains) may anchor to it for settlement and interoperability, but L1 remains the ultimate source of finality.

Encrypted L2. An encrypted L2 is a system in which users submit ciphertext transactions into the base chain, so that ordering and DA are committed to L1 before the contents are revealed. After a coordinated decryption phase, the batch is executed off-chain (e.g., by a zk/optimistic rollup prover), and the L1 rollup contract accepts the new state root only when accompanied by the requisite validity or fraud proof. This preserves L1-level security while mitigating MEV and enabling privacy/fair ordering [36].

2.3 Discrete Logarithm Problem

The *Discrete Logarithm Problem* (DLP) is a cornerstone of modern public-key cryptography. Given a cyclic group $\mathbb{G}$ of prime order q , a generator $g \in \mathbb{G}$, and an element $h \in \mathbb{G}$, the DLP seeks to find value $x \in \mathbb{Z}_q$ such that:

$$g^x = h$$

The DLP is conjectured to be computationally hard in appropriately chosen groups, such as multiplicative subgroups of finite fields $\mathbb{F}$, or elliptic curve groups over finite fields. The security of many cryptographic primitives, including ElGamal encryption [15] relies on the intractability of the DLP.

The difficulty of solving the DLP varies with the group. For generic groups, the best-known algorithm is *Pollard's rho algorithm*, which has an expected running time of $O(\sqrt{q})$ group operations [30]. This remains the best-known generic attack against the DLP and applies equally to both finite fields and elliptic curve groups.

3 Formal Modeling and Definitions

The system consists of a set of parties (*i.e.*, decryptors), denoted $1, \dots, n$, where each party i possesses a (mining) power m_i , such that $\sum_{i=1}^n m_i = W$. We assume parties are rational. So, they aim to maximize their (individual) revenue and may deviate from the prescribed protocol if doing so is advantageous. We model the setup of an encrypted mempool as an n -party protocol

$$\pi: P = \{1, \dots, n\} \rightarrow (A, \tau),$$

where A is a public artifact (*e.g.*, encryption parameters) and τ is a *trapdoor* that enables privileged access to mempool contents before they are committed.

Commit Window. Protocol time is partitioned into discrete *slots*. Each slot corresponds to the strict interval before a set of encrypted transactions is committed (*i.e.*, included and finalized on-chain). We treat slot boundaries as fixed by an external consensus (*e.g.*, the underlying Layer-1's block interval used for data availability). In the context of delayed decryption, this slot length dictates the maximum time window an adversary has to execute a premature decryption before the transactions are safely finalized.

Definition 1 (Computationally Epoch-Easy, Epoch-Moderate, and Epoch-Hard Trapdoors). *Let $C \subseteq P$, and let κ denote the security parameter. We write “within the commit window” to emphasize that success must occur before the epoch boundary.*

Epoch-Easy. *There exists a PPT extractor $\mathcal{E}$ such that*

$$\Pr[\mathcal{E}(\text{state}_C, T) \rightarrow \tau \text{ within the commit window}] \geq 1 - \text{negl}(\kappa).$$

Epoch-Moderate. *There exist a PPT extractor $\mathcal{E}$ and a non-negligible function $\mu(\kappa) \in (0, 1)$ such that*

$$\Pr[\mathcal{E}(\text{state}_C, T) \rightarrow \tau \text{ within the commit window}] \geq \mu(\kappa),$$

yet for every PPT extractor $\mathcal{E}'$ there exists a non-negligible function $\gamma(\kappa) \in (0, 1)$ with

$$\Pr[\mathcal{E}'(\text{state}_C, T) \rightarrow \tau \text{ within the commit window}] \leq 1 - \gamma(\kappa).$$

Intuitively, extraction is sometimes possible with non-negligible probability, but never overwhelmingly likely by any PPT algorithm within a single commit window.

Epoch-Hard. *There exists a function $\text{Adv}(W, D) \in [0, 1]$ such that:*

- *$\text{Adv}(W, D)$ upper-bounds the maximum success probability, over all PPT adversaries/extractors, of producing a valid trapdoor for a puzzle of difficulty D given total work budget W (measured in a fixed unit, *e.g.*, group operations);*
- *for any coalition C (even $|C| = n$) with feasible per-epoch work rate ρ and commit window length L , the probability of recovering τ within the commit window satisfies*

$$\Pr[\mathcal{E}(\text{state}_C, T) \rightarrow \tau \text{ within the commit window}] \leq \text{Adv}(W = \rho L, D) + \text{negl}(\kappa).$$

In particular, any Epoch-Moderate instance is characterized by non-negligible bounds μ, γ satisfying

$$\mu(\kappa) \leq \text{Adv}(\rho L, D) \leq 1 - \gamma(\kappa) \quad \text{for feasible } (\rho, L, D).$$

Easy trapdoors make collusion strictly rational, since coalitions can almost surely recover τ . Epoch-Moderate trapdoors are strictly *less* damaging: coalitions may only extract τ with non-negligible probability, so their expected gain is diluted. Epoch-Hard trapdoors eliminate profitable deviations altogether. Thus, even without slashing or detection, Epoch-Moderate trapdoors are preferable to easy trapdoors in terms of incentive risk.

Recoverability. The preceding definition classifies early opening within a single epoch. We now separate this from a second question: whether the state generated before an epoch gives a reusable shortcut for opening that epoch.

For a target epoch e , let (A_e, τ_e) denote the public artifact and opening trapdoor for that epoch, and let T denote the public transcript available during epoch e 's commit window. Let $\text{state}_{<e}$ denote any state that can arise from the protocol execution before epoch e . This state may be public or private, and may include internal state held by any party or component of the system. The only restriction is that $\text{state}_{<e}$ contains no information first generated in epoch e or later.

Definition 2 (Recoverability). *An encrypted mempool protocol satisfies Recoverability if, for every target epoch e and every PPT extractor $\mathcal{E}$, there exists a PPT extractor $\mathcal{E}'$ with the same feasible work budget during epoch e 's commit window such that, for every past state $\text{state}_{<e}$ that can arise from the protocol execution before epoch e ,*

$$\begin{aligned} \Pr[\mathcal{E}(\text{state}_{<e}, A_e, T) \rightarrow \tau_e \text{ within epoch } e\text{'s commit window}] \\ \leq \Pr[\mathcal{E}'(A_e, T) \rightarrow \tau_e \text{ within epoch } e\text{'s commit window}] \\ + \text{negl}(\kappa). \end{aligned}$$

Remark 1. The order of quantifiers is intentional: $\mathcal{E}'$ may depend on $\mathcal{E}$ and the target epoch e , but not on the particular past state $\text{state}_{<e}$.

Recoverability is independent of the Epoch-Easy, Epoch-Moderate, and Epoch-Hard classification. Those notions measure the probability of early opening within a single epoch. Recoverability measures whether the state generated before an epoch increases the attacker's advantage for opening that epoch. Thus an Epoch-Moderate protocol may still be recoverable, while Epoch-Hard protocol may fail to be recoverable if state carried from earlier protocol execution helps recover future trapdoors.

Definition 3 (Parallelizable, Memoryless Search (PMS)). *An attack model is parallelizable, memoryless if for some $\mathcal{K}(D) > 0$ the success probability after W units of work is $p(W, D) = 1 - \exp(-W/\mathcal{K}(D))$. Equivalently, the model admits fully parallelizable independent trials with constant hazard rate $1/\mathcal{K}(D)$.*

Remark 2 (DecryptChain's DLP puzzle is modeled as PMS). In DecryptChain, once the epoch encryption key ek_i is fixed, recovering the opening trapdoor is exactly the discrete-logarithm instance $g^{\text{dk}_i} = \text{ek}_i$ in the active prime-order group. The key unpredictability mechanism ensures that this instance is fresh: the adversary cannot choose ek_i from a precomputed set for which it already knows the discrete logarithm, except with negligible probability. Thus, within the commit window, the remaining attack is real-time computation on the active DLP instance.

For the prime-order groups used by DecryptChain, the relevant generic attack is Pollard's rho, whose expected work scales as $\tilde{\Theta}(\sqrt{q})$ group operations. This work is parallelizable: independent decryptors can run independent randomized walks, and the success probability depends on the aggregate amount of work spent on the instance, rather than on which identity performed the work. We therefore model aggregate trapdoor recovery for the active DLP instance by the PMS abstraction above, with the scale $\mathcal{K}(D)$ corresponding to the expected generic DLP work at difficulty D .

This is a modeling assumption about the first-solution time of aggregate randomized DLP search. It is the same assumption used later in the pre-execution privacy and liveness analyses, where solving times are treated as exponentially distributed with a calibrated target window.

Theorem 1 (Impracticability of negligible per-epoch breach under parallelizable search). *Fix a commit window length $L = L(\kappa) > 0$ (allowed to depend on the security parameter κ ; typically L is a fixed constant like 12 s or at most $\text{poly}(\kappa)$). Consider any encrypted mempool scheme whose per-epoch security reduces to inverting a fresh instance by parallelizable, memoryless search, i.e., there exists a scale $\mathcal{K}(D) > 0$ (determined by the difficulty parameter D) such that an adversary*

performing total work W within the window succeeds with probability

$$p(W, D) = 1 - \exp\left(-\frac{W}{\mathcal{K}(D)}\right).$$

Let the target window be $T \triangleq \mathcal{K}(D)/R$, where R is the total network work rate (we never use R directly; all bounds are in terms of T), so the worst-case breach probability in one epoch is

$$p_{\text{wc}}(L, T) = 1 - \exp\left(-\frac{L}{T}\right).$$

If the scheme is practical, in the sense that its difficulty grows at most polynomially in κ (equivalently, $T(\kappa) \leq \text{poly}(\kappa)$ for all sufficiently large κ), then it cannot achieve negligible per-epoch breach: for every negligible function $\mu(\kappa)$ there exists κ_0 such that for all $\kappa \geq \kappa_0$,

$$p_{\text{wc}}(L(\kappa), T(\kappa)) > \mu(\kappa).$$

Equivalently, to enforce $p_{\text{wc}}(L, T) \leq \mu(\kappa)$ one must have $T(\kappa) \geq L(\kappa)/\mu(\kappa)$, which is super-polynomial in κ whenever μ is negligible.

Proof. By definition,

$$p_{\text{wc}}(L, T) = 1 - \exp\left(-\frac{L}{T}\right).$$

If we require $p_{\text{wc}}(L, T) \leq \mu(\kappa)$, then $1 - \exp(-L/T) \leq \mu(\kappa)$, so $\exp(-L/T) \geq 1 - \mu(\kappa)$ and hence $L/T \leq -\ln(1 - \mu(\kappa)) \leq 2\mu(\kappa)$ for all large κ (since $-\ln(1 - x) \leq 2x$ for small x). Therefore

$$T(\kappa) \geq \frac{L(\kappa)}{-\ln(1 - \mu(\kappa))} \geq \frac{L(\kappa)}{2\mu(\kappa)}.$$

If $L(\kappa)$ is at most polynomial and $\mu(\kappa)$ is negligible (smaller than any inverse polynomial), the right-hand side is super-polynomial in κ . Thus any scheme with $T(\kappa) \leq \text{poly}(\kappa)$ fails to make p_{wc} negligible for all large κ . Equivalently, achieving negligible per-epoch breach requires $T(\kappa)$ to be super-polynomial, which is impractical by the premise. $\square$

Remark 3. The theorem is *primitive-agnostic* and applies to any design whose best attack behaves like a parallelizable, memoryless search with exponential success curve in total work (hash preimage, generic EC-DLP, etc.). However, it *does not* apply to designs that break parallelism via *sequential* time assumptions (e.g., verifiable delay functions or trapdoor-free timelock encryption in unknown-order groups), nor to designs that rely on committees/TEEs with credible *external* enforcement (e.g., slashing) to deter collusion. In those cases the assumptions change: either the effective work *cannot* be parallelized (so p need not scale like $1 - e^{-L/T}$), or the game-theoretic incentives are altered by enforceable penalties.

On the limits of sequential options. A natural response to the parallelizable-search impossibility is to require sequential work so that throwing hardware at the problem does not reduce latency asymptotically. Two families are typically proposed: (i) time-lock puzzles (TLP) [33], which delay access to some plaintext; and (ii) verifiable delay functions (VDFs) [4] that enforce a public delay before some value can be computed. We argue that neither family, as currently understood, yields a practical, committee-free encrypted mempool. TLP lets an encryptor embed a message that only becomes readable after a delay. This does not match the encrypted mempool interface since:

(a) *Key-centric TLP is impossible without a backdoor.* To time-lock the epoch decryption key, one must *know* that key at encryption time. In a committee-free design, no party knows the key a priori; arranging for someone to embed it creates exactly the operator/vendor backdoor we ruled out.

(b) *Message-centric TLP does not scale.* The fallback is to time-lock *each transaction*. Let each TLP instance require D inherently sequential steps; let the fastest adversarial implementation be at most $\beta \geq 1$ times faster than a reference step rate f (steps/s). Preventing any early read within a window L forces $D > \beta f L$ per transaction. For a block with N transactions, even with M parallel evaluators after the window, the earliest time all transactions can be opened is

$$T_{\text{unlock}} \geq \frac{ND}{M\beta f} > \frac{NL}{M},$$

so the required hardware scales linearly with N , or the post-window latency grows linearly with N . This is incompatible with high-throughput mempools.

VDFs enforce a publicly verifiable delay for a specific computation $y \leftarrow \text{Eval}(x)$, not general decryption. To turn them into a mempool privacy layer, one must either (i) bake decryption into the delayed computation (which again presumes someone embeds a secret up front) or (ii) force each transaction to carry an individualized delayed computation (which inherits the same $O(N)$ scaling as message-centric TLP). We note the Delay Encryption primitive [7] is another sequential candidate that does not enforce individual delayed computation but relies on a trusted setup assumption that essentially negates our design goal.

Consequently, our practical instantiation targets the *Epoch-Moderate* regime that strictly improves over the *Epoch-Easy* trapdoors prevalent in practice: the coalition’s success is bounded away from probability 1 by a non-negligible gap, reducing expected collusive gain within each epoch.

Limitations. Our model isolates collusion incentives around trapdoor extraction during fixed commit epochs. We do not capture other attack vectors such as block withholding or manipulation of base-layer difficulty/epoch timing. These threats are orthogonal to trapdoor reconstruction and require

separate analysis, *e.g.*, at the level of the underlying consensus protocol.

4 The DecryptChain Design

In this section, we present a step-by-step protocol design for DecryptChain and elaborate on its details. A full algorithmic description is given in Algorithm 1, and its workflow visualization is presented in Figure 2.

4.1 System and Threat Model

DecryptChain operates as a permissionless L2 decryption market anchored to a base L1 blockchain. By decoupling block production from decryption, the system requires no trusted setup or designated committee.

In our system, there exist three main entities: (1) **L1 smart contract**, which resides on the base chain (*e.g.*, Ethereum) and serves as the trustless anchor. It handles user transaction ingestion, verifies (zero-knowledge) validity proofs for L2 state updates, enforces the smooth difficulty adjustment, and dispenses block rewards. (2) **L1 users**, who encrypt their transactions under the current epoch’s public encryption key (ek_i) and submit them to the smart contract, protecting their execution from MEV until the batch is committed. (3) **Decryptors**, who are L2 miners competing to solve the current epoch’s discrete logarithm puzzle. A successful miner recovers the decryption key (dk_i), locally decrypts the committed batch, and submits a verifiable state update to the smart contract.

Threat model. We assume the underlying L1 chain provides robust data availability and consensus. We consider a static Probabilistic Polynomial-Time (PPT) adversary who can compromise a subset of miners and all users except for one. Our primary evaluation of the cryptographic bounds for premature trapdoor recovery was established in Section 3. We require four main security properties for DecryptChain, including *Epoch Key Unpredictability*, *Pre-execution Privacy*, *Guaranteed Decryption*, and *Recoverability*, which are all formally defined in Appendix A.

4.2 Our Construction

Before proceeding with details, we give a high-level description of our system: At each epoch (block interval), DecryptChain publishes an (Elliptic Curve ElGamal) encryption key $ek_i = dk_i G$ (or $ek_i = g^{dk_i}$ with multiplicative notation) to let users on L1 base chain encrypt their transactions. Miners then compete to recover the corresponding decryption key by solving a DL instance.

A proper realization of the aforementioned system requires careful protocol design, which we elaborate on next.

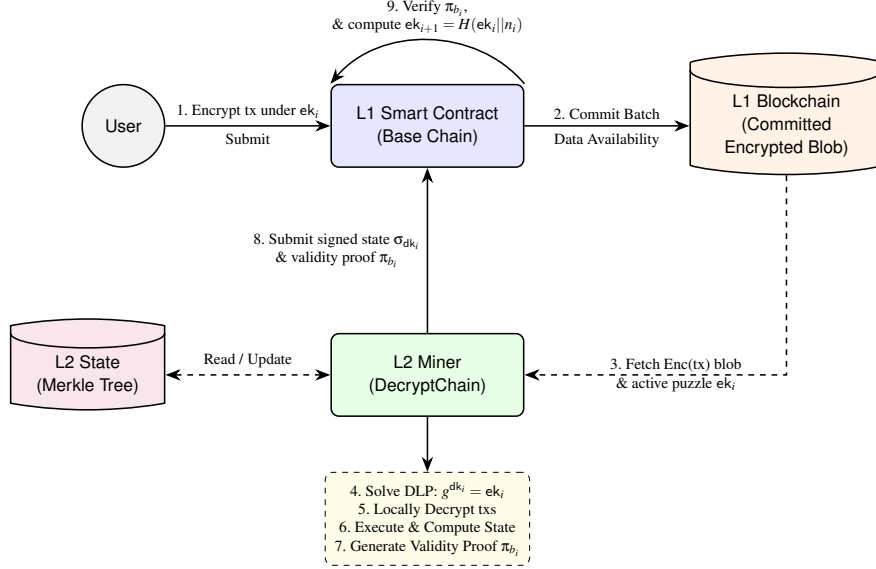


Figure 2: DecryptChain work flow.

① Computing the encryption key. In order to let a user on the L1 chain encrypt their transaction tx , we need to devise a method to securely generate an encryption key ek . Security here implies that no one knows the corresponding decryption key dk until a predetermined time T . It is worth highlighting that the need to adopt a public-key encryption stems from the asymmetry between a public-private key pair, such that one can (computationally) be derived from the other.

Due to the nature of solving a DL instance as a computationally hard problem, targeting an expected constant rate for a solution (*i.e.*, key extraction) is the best we can hope for. This offers a regular time interval for decryption (and also key generation), which is crucial for the stability of the system in a similar fashion to the typical Bitcoin PoW hash puzzle. Given that a DL-based puzzle is a structured hard problem (unlike a hash puzzle), we need to enforce independence and unpredictability of the instances via our design to achieve an exponential distribution for block arrival time by relying on the honest majority assumption of total computing power.⁴ Moreover, we would like to have a *smoothly tunable* difficulty adjustment to be able to adapt the key extraction time depending on the computation power in the system. To do so, let assume each miner i who has solved the current puzzle (*i.e.*, extracting decryption key dk_i), sample a fresh, independent nonce n_i to determine the next encryption key ek_{i+1} as a group element on an elliptic curve by computing a hash-to-curve $H(ek_i || n_i)$ modeled as random oracle (we later show how to choose the proper curve description).⁵ This

⁴If miners use different algorithms or tuned implementations, the variance of solve-time inevitably could be higher. However, we assume miners follow the best-known approach for solving the puzzles.

⁵In a properly chosen curve with prime-order subgroup, there exists a private key for a chosen public key due to injectivity.

ensures that DL instances are chained and independent from each other, preventing pre-computation, *i.e.*, binding instances to recent entropy so they cannot be prepared in advance. One caveat is that there is no way to know if the miner indeed chooses a fresh nonce (unless assuming the existence of public randomness [23]). Note that if the miner can control the input of the random oracle, they can program it to output a certain curve point for which they already know the DL via pre-computation. We can tackle this issue by having the commitment (*i.e.*, Merkle root) of the decrypted transactions as the nonce to guarantee unpredictability of the next public encryption key given one honest contribution involved. This, however, causes another issue: the miner who obtains the current decryption key dk_i can *privately* work towards mining the next decryption key dk_{i+1} , enabling premature decryption. Our idea is to consider the most recent L1 block hash as a nonce, including the winning miner’s transaction. This essentially binds the computation of the next encryption key to the disclosure of the miner’s transaction (including the decrypted blob). This then puts them under the threat of having another miner take up their efforts by posting their transaction on L1 earlier if they refrain from doing so.

② Finding the decryption key. At each block interval i , the miners compete to crack the target encryption key until they find the corresponding decryption key dk_i . This should take an expected constant amount of time T according to the desired difficulty level of the DecryptChain. A miner who solves the puzzle first should propose the decryption key dk_i to L1 contract to receive their reward after verification. However, this makes them vulnerable to the possibility of front-running by others who wish to claim the key extraction

reward. To solve this problem, the winning miner on DecryptChain does not explicitly expose the decryption key after mining but instead *locally* decrypts the encrypted transactions and sends back the *signed decrypted blob* together with a validity proof. This essentially solves the frontrunning problem since the signature with the mined (ephemeral) decryption key shows the authenticity of the winner with respect to the current encryption key ek_i without anyone being able to learn the decryption key or forge the signature.⁶ However, submitting a full decrypted blob might be gas-heavy for the miner and economically expensive. A workaround would be to submit *signed state update commitment* σ_{dk_i} (i.e., the new Merkle root of the DecryptChain state after applying the decrypted blob) and a succinct validity proof π_{b_i} attesting the correctness of the state update. Crucially, to bind this L2 execution to the L1 data availability layer, the smart contract passes the cryptographic commitment of the previously committed ciphertext blob (e.g., a Merkle root or KZG commitment) as a public input to the ZK verifier, alongside the current state root of DecryptChain.

③ On-chain verification. Given that DecryptChain is an L2 chain, it runs on top of an underlying L1 chain (e.g., Ethereum) as its anchor of trust. The connection between DecryptChain and its L1 chain is through its deployed smart contract that handles tasks such as receiving encrypted transactions from L1 users. Crucially, having this contract on L1 allows users to (verifiably) get to know about the latest state on L2 and vice versa. In particular, DecryptChain contract does the following: (1) verifying the signature σ_{dk_i} with respect to ek_i , (2) verifying the proof π_{b_i} for state update with respect to the current state and encrypted blob, (3) computing the next encryption key $ek_{i+1} = H(ek_i || n_i)$, where n_i is latest block hash on L1. Observe that since the contract does not have access to the decryption key dk_i , it uses the current encryption key together with the nonce to derive ek_{i+1} , preserving the independence and unpredictability of the puzzles and keeping them chained. Moreover, DecryptChain smart contract also needs to choose a secure curve with proper order for encryption key generation and support a smooth difficulty adjustment to ensure that key extraction/block times remain stable despite improvements in computational methods or hardware. To handle this efficiently on-chain, we let the (designated)⁷ miner provides the curve description to the L1 contract together with accompanying validity proof π_D that we later elaborate on in Section 4.4.

⁶Ensuring that whoever first submits a valid solution wins is a known problem on public blockchains like Ethereum. The most common solution is based on “commit-reveal” and other relevant timing mechanisms.

⁷Note that for all the decryption keys mined in the middle (i.e., between each difficulty adjustment window) there is no need for such proof. So, this allows the system to be flexible in choosing whom to provide this proof.

Algorithm 1 DecryptChain Protocol

Require: Initial key ek_0 , target interval T , retarget window w . Miners generate L1 keys (sk_m, pk_m) .

```

1: Initialization:
2:  $ek \leftarrow ek_0; i \leftarrow 0$  ▷ Set genesis parameters
3: Deploy L1 smart contract (SC) and start the first puzzle

L2 Miner: Executes continuously for the active epoch  $i$ .
4: procedure FINDDECRYPTIONKEY( $ek_i$ )
5:   loop
6:     if miner finds  $dk_i$  such that  $g^{dk_i} = ek_i$  then
7:        $b_i, s_i \leftarrow \text{LocallyDecryptAndExecute}(txs, dk_i)$ 
       ▷ Decrypt & compute new state
8:        $\sigma_{dk_i} \leftarrow \text{Sign}(dk_i, s_i)$  ▷ Sign state update
9:        $\pi_{b_i} \leftarrow \text{Prove}(dk_i, c_i, b_i, s_i, pk_m)$  ▷ Generate ZK
       validity proof
10:      if  $i \pmod w == 0$  then compute new curve
        $D$  and proof  $\pi_D$ 
11:      Submit  $(\sigma_{dk_i}, \pi_{b_i}, [D, \pi_D])$  to SC
12:      return success
13:    end if
14:  end loop
15: end procedure
```

L1 Smart Contract: Triggered upon submission by a winning miner.

```

16: procedure VERIFYANDPROCEED( $\sigma_{dk_i}, \pi_{b_i}, \{D, \pi_D\}$ )
17:   VerifyProof( $\pi_{b_i}$ ) ▷ Verifies state update, DA commitment,
   and  $\sigma_{dk_i}$ 
18:    $n_i \leftarrow \text{GetLatestL1BlockHash}()$  ▷ Source of
   unpredictability
19:    $ek_{i+1} \leftarrow H(ek_i || n_i)$  ▷ Derive next encryption key
20:   Update  $ek \leftarrow ek_{i+1}; i \leftarrow i + 1$  ▷ Advance epoch
21:   Send block reward to the submitter's  $pk_m$ 
22:   if  $i \pmod w == 0$  then ▷ Trigger smooth retargeting
23:      $(N_{new}, N_{min}, N_{max}) \leftarrow \text{Run Algorithm 2}$ 
24:     if VerifyProof( $\pi_D$ ) and  $D.\text{order} \in [N_{min}, N_{max}]$ 
25:       then
       Update protocol parameters; set  $D$  as the new
       curve
26:     else
27:       Reject submission ▷ Miner loses reward; state
       update reverts
28:     end if
29:   end if
30: end procedure
```

Putting it all together. With these steps in place, including ① secure encryption key generation with an expected constant decryption key extraction time, ② verifiable key extraction and state update, and ③ verifiable curve selection with smooth difficulty adjustment, DecryptChain presents a fully fledged encrypted mempool for its underlying L1 chain. For each

block interval time T , users on L1 encrypt their transactions under ek_i and send them to L1 contract; the miners on DecryptChain compete to find the corresponding decryption key dk_i ; and the winning miner decrypts the (committed) blob of encrypted transactions and presents the signed state update commitment together with a validity proof to the contract and gets rewarded. As an extension, we provide a *universal variant* for our DecryptChain in appendix D, where it *simultaneously* supports multiple underlying chains.

Theorem 2. *The DecryptChain protocol satisfies Epoch Key Unpredictability, Pre-execution Privacy, Guaranteed Decryption, and Recoverability.*

Proof. We provide the full security analysis and proofs in Appendix A. $\square$

4.3 Probabilistic Bounds of Early Decryption

One vital subtlety is to handle the discrepancy between the block commit time on L1 and the key extraction time on DecryptChain. That is, while L1 may have a fixed/deterministic block interval time for committing to the set of encrypted transactions (e.g., 12-second block production time on Ethereum), the extraction time on DecryptChain is inherently probabilistic with an exponential distribution. This implies a probabilistic analysis to understand what is the chance of having a miner on DecryptChain obtain the decryption key prior to transactions being committed on L1.

Attacker work model. We quantify an adversary by the *aggregate* feasible work rate it can mobilize during an epoch’s commit window of length L . Under a parallelizable, memory-less search (PMS; Definition 3), success depends only on the ratio of time budget to the problem’s *target window* T (the expected time for someone to solve and post the answer under the current difficulty). Thus, the per-epoch breach probability is

$$p_{\text{breach}}(L, T) = 1 - e^{-L/T}.$$

Remark 4 (on attacker capacity modeling). A realistic model should capture the *entire* discrete-log capacity a rational coalition can elastically aggregate (pooling). The PMS abstraction already subsumes this: the effect of aggregate work appears only through the hazard $1/T$, so our analysis uses (L, T) directly rather than fixing the attacker to a static fraction of global capacity.

We instantiate the setup as follows:

Trapdoor and public artifact. In a prime-order group $\mathbb{G}$, epoch e uses keypair (dk_e, ek_e) with $ek_e = dk_e G$. The trapdoor τ is dk_e ; the public artifact A includes encryption key ek_e , and group parameters.

Commit window of Epoch. The model’s epoch involves an initial commit window; users encrypt transactions to ek_e and submit.

Table 2: Per-epoch breach probability $p_{\text{breach}} = 1 - e^{-L/T}$ for commit window $L = 12$ s and various target windows T . Probabilities are expressed as powers of two.

L (s)	Target window T	p_{breach}
12	1 minute	$2^{-2.46}$
12	10 minutes	$2^{-5.65}$
12	1 hour	$2^{-8.23}$
12	1 day	$2^{-12.82}$
12	1 week	$2^{-15.61}$
12	1 month	$2^{-17.72}$
12	1 year	$2^{-21.34}$
12	100 years	$2^{-28.10}$
12	1000 years	$2^{-31.44}$

Targeted reveal. After the commit ends, a targeted reveal window opens in which some miner computes dk_e .

Difficulty adjustment sets the effective problem scale so that the expected time-to-solution is T ; hence the only tunable security-latency knobs are the commit window L and the target window T . In the notation of Definition 1, the epoch success bound is $\mathcal{A}(W, D) = p_{\text{breach}}(L, T) = 1 - e^{-L/T}$.

Table 2 reports $p_{\text{breach}}(L, T)$ for a fixed commit window $L = 12$ s (Ethereum-like) and a range of target windows T (1 minute up to 1000 years). We express probabilities as powers of two. For example, at $T = 10$ minutes,

$$p_{\text{breach}}(12 \text{ s}, 600 \text{ s}) = 1 - e^{-12/600} \approx 2^{-5.65} (\approx 1.98\% \text{ per epoch}).$$

As T increases, p_{breach} decays roughly like L/T . Achieving a *negligible* target such as $p_{\text{breach}} \leq 2^{-40}$ at $L = 12$ s would require

$$T \gtrsim \frac{L}{2^{-40}} = L \cdot 2^{40} \approx 4.2 \times 10^5 \text{ years},$$

which is impractical. This matches Theorem 1: under PMS, negligible per-epoch breach at realistic L forces astronomically large effective delay T .

Theorem 3 (DecryptChain is Epoch-Hard under negligible per-epoch breach). *If the commit and target windows (L, T) satisfy*

$$p_{\text{breach}}(L, T) = 1 - e^{-L/T} \leq \text{negl}(\kappa),$$

then, by Definition 1, DecryptChain is Epoch-Hard.

Proof. The proof is immediate from the definition of Epoch-Hard with $\mathcal{A}(W, D) = p_{\text{breach}}(L, T)$. $\square$

Theorem 4 (DecryptChain is Epoch-Moderate under practical commit and time windows). *For practical deployments with protocol-level commit windows in the seconds range (e.g.,*

$L = 12$ s) and difficulty producing realistic target windows T in minutes, DecryptChain is Epoch-Moderate: there exist non-negligible functions μ, γ such that

$$\mu(\kappa) \leq p_{\text{breach}}(L, T) = 1 - e^{-L/T} \leq 1 - \gamma(\kappa).$$

Numerically, Table 2 shows $p_{\text{breach}} \in [2^{-2.46}, 2^{-12.82}]$ for T from 1 minute at $L = 12$ s, i.e., non-negligible per-epoch success.

Proof. By Theorem 1, enforcing $1 - e^{-L/T} \leq \text{negl}(\kappa)$ at realistic L would require astronomically large T (super-polynomial effective delay). Therefore, for practical (L, T) , $p_{\text{breach}}(L, T)$ is non-negligible yet bounded away from 1, which is exactly the Epoch-Moderate regime (Definition 1). The values in Table 2 substantiate this for $L = 12$ s. $\square$

4.4 Difficulty Adjustment

A key subtlety in the design of DecryptChain is to ensure that difficulty retargeting is *smooth*, so the network maintains a relatively stable solve/block time. We call an adjustment *smooth* when parameter updates are gradual across retarget windows rather than abrupt. Retargets occur at fixed windows (e.g., every w valid extraction), which reduces short-term fluctuations and improves chain stability.⁸ We now present the full details of our smooth difficulty adjustment algorithm.

Choosing a secure curve. We require proofs that the active group is prime-order (or cofactor 1 with enforced subgroup checks) and free of exploitable structure; otherwise, the discrete-log hardness model may not apply. For generic attacks such as Pollard’s ρ , the expected work to recover a secret scales as $\tilde{\Theta}(\sqrt{N})$ in the group order N . Tuning N across families of *secure* curves therefore yields a predictable, *smooth* hardness gradient; tuning via insecure subgroups does not.

Given a target prime order n , miners can very quickly (polylogarithmic in $\log n$) produce a matching secure curve using a standard CM-based routine with tiny discriminants: solve a small norm equation, test one of two candidate field sizes for primality, evaluate a class polynomial modulo that field, and instantiate the curve (or its twist). In practice, this runs in milliseconds–seconds on commodity hardware, so any miner can compute it locally at retarget time (full reference code is given in Appendix C).

Smart contract role. On-chain curve search is expensive; instead, the contract exposes a permissionless interface where, at each retarget boundary, any party proposes a new curve together with a succinct proof that (a) the curve is well-formed and has prime order N , and (b)

N lies within the acceptance band determined by the current target. The contract accepts the curve iff the proof verifies against on-chain parameters. We detail the construction of this unified succinct proof for curve sanity, primality, and cofactor exclusion in Appendix B.

Square-root control law. Let S^* denote the target seconds per solve and w the retarget window (solves per window). The target window time is $T^* := wS^*$. If the previous window ran at target order N_{old} and finished in observed time T_{obs} , then under the generic DL model

$$\mathbb{E}[t] \approx c\sqrt{N}, \quad T \approx wc\sqrt{N}, \quad (1)$$

for constant $c > 0$. Estimating $c \approx T_{\text{obs}}/(w\sqrt{N_{\text{old}}})$ and enforcing $w c \sqrt{N_{\text{new}}} = T^*$ gives the *square-root retarget rule*:

$$N_{\text{new}} = N_{\text{old}} \cdot \left(\frac{T^*}{T_{\text{obs}}} \right)^2. \quad (2)$$

Thus, if the last window was 10% slower ($T_{\text{obs}} = 1.1T^*$), we reduce the target order by a factor $(1/1.1)^2 \approx 0.826$ (i.e., $\approx 17.4\%$ smaller). If it was 10% faster ($T_{\text{obs}} = 0.9T^*$), we increase by $(1/0.9)^2 \approx 1.234$ (i.e., $\approx 23.4\%$ larger). This quadratic adjustment is the correct analogue of linear hash-rate retargeting in hash-based PoW, reflecting that DL cost scales with $\sqrt{N}$ rather than N .

Stability, acceptance band, and manipulation resistance. For stability we clamp the speed ratio $r := T^*/T_{\text{obs}}$ to $[r_{\min}, r_{\max}]$ (e.g., $r_{\min} = 1/2, r_{\max} = 2$), implying $N_{\text{new}}/N_{\text{old}} \in [r_{\min}^2, r_{\max}^2]$. The contract then derives an acceptance band

$$N \in [(1 - \tau)N_{\text{new}}, (1 + \tau)N_{\text{new}}]$$

with tolerance τ (e.g., $\tau = 5\%$). Only curves whose *proved* order N lies in this band are admitted. To limit gaming (delaying reveals to inflate T_{obs}), we seal the epoch at first valid commit, may impose reveal deadlines or latency-decayed rewards, and rely on large W or robust estimators to reduce variance.

Algorithm 2 Square-Root Difficulty Retarget

Require: Target seconds per solve $S^* > 0$; window size $w \in \mathbb{N}$; previous target order $N_{\text{old}} \geq 1$; observed window time $T_{\text{obs}} > 0$; clamp $0 < r_{\min} \leq 1 \leq r_{\max}$; tolerance $\tau \in [0, 1]$.

Ensure: New target order N_{new} and acceptance band $[N_{\min}, N_{\max}]$.

- 1: $T^* \leftarrow w \cdot S^*$ $\triangleright$ target batch time
 - 2: $r \leftarrow T^*/T_{\text{obs}}$ $\triangleright$ speed ratio (target / observed)
 - 3: $r \leftarrow \min\{\max\{r, r_{\min}\}, r_{\max}\}$ $\triangleright$ clamp for smoothness
 - 4: $f \leftarrow r^2$ $\triangleright$ square-root law: DL cost $\propto \sqrt{N}$
 - 5: $N_{\text{new}} \leftarrow \text{round}(N_{\text{old}} \cdot f)$
 - 6: $N_{\min} \leftarrow \lfloor (1 - \tau)N_{\text{new}} \rfloor$; $N_{\max} \leftarrow \lceil (1 + \tau)N_{\text{new}} \rceil$
 - 7: **return** $N_{\text{new}}, N_{\min}, N_{\max}$
-

⁸In Bitcoin PoW, every 2,016 blocks (roughly two weeks) the network adjusts difficulty toward a 10-minute target.

5 Discussions

Claiming rewards and signature verification. When a miner solves the discrete-log puzzle for epoch i , it must *claim* the reward by proving (1) that it correctly executed the epoch’s (now decrypted) transactions and (2) that it knows the decryption key dk_i corresponding to the encryption key $ek = g^{dk_i}$ (over the puzzle’s curve/group). Ethereum, however, only supports a limited set of elliptic-curve primitives (*e.g.*, BN254 precompiles) and does not verify signatures over arbitrary curves. Observe that here we do not necessarily require a separate on-chain signature check. Instead, signature correctness can be enforced *inside* the validity proof, where the (zero-knowledge) circuit both verifies a Schnorr-like signature $\sigma = \text{Sign}_{dk_i}(pk_i)$ and proves that the post-state is the result of deterministically executing the plaintext transactions for that epoch. By offloading the arbitrary-curve signature verification and transaction execution to the ZK circuit, the L1 contract only needs to verify a single succinct proof (*e.g.*, Groth16 or PLONK). On Ethereum, this verification incurs a constant overhead of roughly 300,000 gas, making the on-chain cost highly efficient and entirely independent of the batch size or the chosen puzzle curve.

Off-chain power and silent adversary. A nuance of utilizing an externalized PoW decryption market is the potential for a *silent adversary*. Such an actor might solve the active discrete-log puzzle off-chain and withhold the corresponding opening information. Silence is economically meaningful only when the adversary recovers the decryption key before the epoch’s commit window closes and can use the plaintexts for MEV. If the puzzle is solved only after the encrypted transactions have already been committed in a fixed order on the base chain, then the solver no longer obtains an early-opening advantage for that batch. Its remaining in-protocol benefit is to publish a valid state update and claim the reward.

This observation gives honest participation an important disciplining effect. As more honest parties contribute work, a solver who withholds a solution faces a growing risk that an honest party will publish first and collect the reward. Thus, whenever the private value of silence is small or has already expired, the reward mechanism pushes solvers toward publication: a would-be silent solver becomes observable by claiming the reward before another participant does. Once published, the solution becomes part of the public transcript, liveness is preserved, and the observed latency can be used by the protocol’s difficulty adjustment. In this sense, honest work does not merely compete with hidden work; it also creates pressure for hidden work to reveal itself.

The remaining hidden-power concern is therefore the case in which silence has private value. A silent adversary may solve before the commit window closes, extract MEV from early plaintext access, and then choose not to publish. More strategically, such an adversary might even forfeit an immedi-

ate protocol reward in order to keep its computational capacity unobserved, or even influence future difficulty calibration. This behavior requires a more detailed cryptoeconomic analysis of dynamic rewards and incentives [19], which we leave as an important direction for future work.

We highlight that recoverability limits the damage of such behavior. Even if a silent adversary successfully opens one epoch early, it learns only that epoch’s opening information. It does not obtain a reusable opening capability for future epochs. Every later epoch is a fresh public work instance, so the adversary must again spend real-time computation and again recover the corresponding trapdoor before the commit window closes. More importantly, honest parties can enter later epochs; if they suspect silent MEV extraction,⁹ they can respond in-protocol by adding or subsidizing honest computational power, increasing both the probability that honest solvers win later commit-window races and the pressure on silent solvers to publish before losing the reward.

6 Related Work

Recent years have seen efforts to develop different variants of encrypted mempools, with a focus on combating MEV [8, 9, 12, 22, 28]. The closest work to ours is due to Doweck and Eyal [12], where they put forward the notion of multi-party timed commitment (MPTC). This allows a set of parties to commit to their values without anyone learning the individual ones until after a certain time. They then build Capsule Chain as a PoW blockchain with integrated privacy based on solving DL puzzles. There are major differences, however, between DecryptChain and Capsule Chain. First, it requires users to connect with a designated miner directly due to their interactive communication model instead of sending transactions to the peer-to-peer network, violating the liveness due to abortion and the possibility of having the miner’s work wasted if a user deviates from protocol execution. The other limitation of [12] is tying the underlying consensus mechanism with privacy that has negative impacts on the performance of the system (*e.g.*, block commit time) and demands an elaborated mechanism design (*e.g.*, to handle fees and liveness). Moreover, their puzzle generation does not support smooth difficulty adjustment, given the need for an exponentially large mapping between difficulty parameters and group orders, which is not practical to maintain in a smart contract. Importantly, decoupling consensus from privacy in DecryptChain has multiple security and usability advantages. It allows, among others, using our chain as an application-agnostic solution operating entirely at the application layer with no changes to the underlying blockchain or its consensus protocol. Consequently, it paves the way for unlocking applications like efficient time-lock encryption [18] and integration with data availability techniques [17].

⁹This can be done using on-chain MEV heuristics [31].

FairPoS [8] is another closely related work to ours that targets MEV/front-running by enforcing *input fairness*: an adversary should not be able to decrypt an honestly encrypted input before the block containing it lies in the k -common prefix, and honest parties should eventually decrypt finalized inputs. FairPoS achieves this by integrating *delay encryption* [7] into a longest-chain PoS execution model. FairPoS and our work share the same high-level goal but differ in where the “timely decryption” responsibility lives. FairPoS makes decryption *part of the chain rule*, where leaders must carry forward key-extraction artifacts, and chain validity/extendability enforces the schedule. By contrast, we model encrypted mempools through the lens of *trapdoor availability*: coalitions deviate if premature access is profitable and undetectable. DecryptChain framing externalizes who decrypt into an open, incentive-driven decryption market, rather than requiring consensus participants themselves to reliably perform the work.

At first glance, FairPoS appears *Epoch-Hard*: encrypted inputs are committed to the chain before they are revealed, and the only apparent way to learn plaintext early is to run a delay-extraction procedure that is intended to require (at least) d sequential steps; moreover, FairPoS’s execution model explicitly idealizes away aggregate-work speedups by constraining each participant (including the adversary) to perform only a single extraction step per pending key per round. If we relax this idealization to reflect heterogeneous resources, the picture shifts toward *Epoch-Moderate*: a well-provisioned coalition can plausibly gain a head start via faster implementations/specialized hardware, and in any case the protocol’s decryption is scheduled to occur only after some delay, creating an inherent “who-learns-first” gap between parties that can extract keys as soon as possible and parties that wait for the on-chain key publications [8]. However, under the concrete instantiation of delay encryption [7] used to motivate FairPoS, the construction itself relies on a one-time trusted setup; if that setup is compromised (*i.e.*, an adversary retains the setup trapdoor), then the intended sequential-delay barrier can collapse and premature decryption becomes effectively immediate, placing the scheme in the *Epoch-Easy* regime under our trapdoor taxonomy. Last but not least, the scheme suffers from linear scalability costs, as validators are forced to perform a computationally expensive bilinear pairing for every single transaction ($O(N)$ complexity), creating a verification bottleneck for high-throughput networks compared to standard signature verification. FIRST [34] mitigates front-running at the application layer by requiring users to solve a VDF off-chain, the validity of which is attested by an external committee of solvers to enable gas-efficient on-chain verification. While this circumvents the need for consensus-level upgrades, it significantly degrades the user experience by placing a heavy computational burden on client devices to generate proofs prior to submission. Furthermore, the protocol introduces significant trust assumptions by relying on the honesty and liveness of this off-chain committee, effec-

tively replacing the blockchain’s trustless execution model with reliance on a semi-trusted third party.

7 Conclusion and Future Work

In this paper, we presented a formal taxonomy for evaluating confidentiality failures in commit-reveal encrypted mempools, explicitly isolating the immediate risk of premature decryption from the systemic capability to recover from compromised states. To resolve the structural vulnerabilities inherent in pre-distributed decryption capabilities, we introduced DecryptChain, a novel permissionless, proof-of-work encrypted mempool. Among other properties, DecryptChain ensures that adversarial advantages remain strictly bounded, contestable, and automatically recoverable without out-of-band coordination. Our architecture opens two avenues for future research:

- Recoverability in alternative architectures:** Exploring how threshold or committee-based designs might be fundamentally restructured to achieve the (autonomous) recoverability formalized in our model.
- Cryptoeconomic analysis:** Formalizing the game-theoretic equilibrium between epoch-local mining rewards, difficulty adjustments, and extractable MEV to ensure solver incentives remain overwhelmingly aligned with honest, timely execution.

References

- [1] Amit Agarwal, Rex Fernando, and Benny Pinkas. Efficiently-thresholdizable batched identity based encryption, with applications. In *Annual International Cryptology Conference*, pages 69–100. Springer, 2025.
- [2] Renas Bacho and Alireza Kavousi. Sok: Dlog-based distributed key generation. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 614–632. IEEE, 2025.
- [3] Joseph Bebel and Dev Ojha. Ferveo: Threshold decryption for mempool privacy in bft networks. *Cryptology ePrint Archive*, 2022.
- [4] Dan Boneh, Joseph Bonneau, Benedikt Bünz, and Ben Fisch. Verifiable delay functions. In *Annual international cryptology conference*, pages 757–788. Springer, 2018.
- [5] Dan Boneh, Aditi Partap, and Lior Rotem. Accountability for misbehavior in threshold decryption via threshold traitor tracing. In *Annual International Cryptology Conference*, pages 317–351. Springer, 2024.
- [6] Reinier Broker and Peter Stevenhagen. Constructing elliptic curves of prime order, 2007.

- [7] Jeffrey Burdges and Luca De Feo. Delay encryption. In *Advances in Cryptology—EUROCRYPT 2021: 40th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, October 17–21, 2021, Proceedings, Part I*, pages 302–326. Springer, 2021.
- [8] James Hsin-yu Chiang, Bernardo David, Ittay Eyal, and Tiantian Gong. Fairpos: Input fairness in permissionless consensus. In *5th Conference on Advances in Financial Technologies (AFT 2023)*, pages 10–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2023.
- [9] Arka Rai Choudhuri, Sanjam Garg, Julien Piet, and Guru-Vamsi Policharla. Mempool privacy via batched threshold encryption: Attacks and defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 3513–3529, 2024.
- [10] Arka Rai Choudhuri, Sanjam Garg, Guru-Vamsi Policharla, and Mingyuan Wang. Practical mempool privacy via one-time setup batched threshold encryption. *Cryptology ePrint Archive*, 2024.
- [11] Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, and Ari Juels. Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 910–927. IEEE, 2020.
- [12] Yael Doweck and Ittay Eyal. Multi-party timed commitments. *arXiv preprint arXiv:2005.04883*, 2020.
- [13] Stefan Dziembowski, Sebastian Faust, Tomasz Lizurej, and Marcin Mielniczuk. Secret sharing with snitching. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 840–853, 2024.
- [14] Stefan Dziembowski, Sebastian Faust, and Jannik Luhn. Shutter network: Private transactions from threshold cryptography. *Cryptology ePrint Archive*, 2024.
- [15] Taher ElGamal. A public key cryptosystem and a signature scheme based on discrete logarithms. *IEEE Transactions on Information Theory*, 31(4):469–472, 1985.
- [16] Armando Faz-Hernández, S. Scott, Nick Sullivan, Riad S. Wahby, and Christopher A. Wood. Hashing to elliptic curves. RFC 9380 (IRTF CFRG), August 2023.
- [17] Ben Fisch, Arthur Lazzaretti, Zeyu Liu, and Lei Yang. Permissionless verifiable information dispersal (data availability for bitcoin rollups). In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 1983–2001. IEEE, 2025.
- [18] Nicolas Gailly, Kelsey Melissaris, and Yolan Romain. tlock: practical timelock encryption from threshold bls. *Cryptology ePrint Archive*, 2023.
- [19] Pranav Garimidi, Lioba Heimbach, and Tim Roughgarden. Transaction fee mechanism design for leaderless blockchain protocols. In *International Conference on Financial Cryptography and Data Security*, pages 20–35. Springer, 2025.
- [20] Helmut Hasse. Zur theorie der abstrakten elliptischen funktionenkörper iii. die struktur des meromorphismenrings. die riemannsche vermutung. *Journal für die reine und angewandte Mathematik*, 1936(175):193–208, 1936.
- [21] Lioba Heimbach and Roger Wattenhofer. Sok: Preventing transaction reordering manipulations in decentralized finance. In *4th ACM Conference on Advances in Financial Technologies (AFT)*, 2022.
- [22] Alireza Kavousi, Duc V Le, Philipp Jovanovic, and George Danezis. Blindperm: Efficient mev mitigation with an encrypted mempool and permutation. In *29th International Conference on Principles of Distributed Systems*, 2026.
- [23] Alireza Kavousi, Zhipeng Wang, and Philipp Jovanovic. Sok: public randomness. In *2024 IEEE 9th European Symposium on Security and Privacy (EuroS&P)*, pages 216–234. IEEE, 2024.
- [24] Mahimna Kelkar, Aadityan Ganesh, Aditi Partap, Joseph Bonneau, and S Matthew Weinberg. Breaking omertà: On threshold cryptography, smart collusion, and whistleblowing. *Cryptology ePrint Archive*, 2025.
- [25] Amirhossein Khajepour, Hanzaleh Akbarinodehi, Mohammad Jahanara, and Chen Feng. A mempool encryption scheme for ethereum via multiparty delay encryption. *Cryptology ePrint Archive*, 2023.
- [26] Gary L. Miller. Riemann’s hypothesis and tests for primality. *Journal of Computer and System Sciences*, 13(3):300–317, 1976.
- [27] R Miller. Mev-sgx: A sealed bid mev auction design, 2021.
- [28] Jonathan Passerat-Palmbach. Leveraging homomorphic encryption for maximally extractable value (mev) mitigation: Enabling blind arbitrage on decentralised exchanges.
- [29] Stephen C. Pohlig and Martin E. Hellman. An improved algorithm for computing logarithms over $GF(p)$ and its cryptographic significance. *IEEE Transactions on Information Theory*, 24(1):106–110, 1978.

- [30] John M Pollard. Monte carlo methods for index computation (mod p). *Mathematics of computation*, 32(143):918–924, 1978.
- [31] Kaihua Qin, Liyi Zhou, and Arthur Gervais. Quantifying blockchain extractable value: How dark is the forest? In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 198–214. IEEE, 2022.
- [32] Michael O. Rabin. Probabilistic algorithm for testing primality. *Journal of Number Theory*, 12(1):128–138, 1980.
- [33] Ronald L Rivest, Adi Shamir, and David A Wagner. Time-lock puzzles and timed-release crypto. 1996.
- [34] Emrah Sariboz, Gaurav Panwar, Roopa Vishwanathan, and Satyajayant Misra. First: Frontrunning resistant smart contracts. In *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*, pages 873–889, 2025.
- [35] Stephan Van Schaik, Alex Seto, Thomas Yurek, Adam Batori, Bader AlBassam, Daniel Genkin, Andrew Miller, Eyal Ronen, Yuval Yarom, and Christina Garman. Sok: Sgx. fail: How stuff gets exposed. In *2024 IEEE symposium on security and privacy (SP)*, pages 4143–4162. IEEE, 2024.
- [36] Guy Zyskind et al. Fhe-rollups: Scaling confidential smart contracts on blockchains. In *ACM Advances in Financial Technologies (AFT 2024/2025)*, 2024.

A Formal Security Properties

Before we formally define and analyse the security properties of DecryptChain, we establish the mathematical soundness of the dynamic key generation.

Injectivity of DecryptChain Puzzles . Let $\mathbb{G} = \langle g \rangle$ be an elliptic curve group of prime order q . The scalar-multiplication map $f : \mathbb{Z}_q \rightarrow \mathbb{G}$ defined by $f(x) = g^x$ is a group isomorphism, and hence bijective. Consequently, for any valid epoch encryption key $ek_i \in \mathbb{G}$ generated by the protocol, there exists exactly one unique decryption key $dk_i \in \mathbb{Z}_q$ such that $g^{dk_i} = ek_i$.

This injectivity guarantees that once a solution is found, it is the *only* solution, preventing an adversary from submitting a secondary, malicious trapdoor. Next, we prove that an adversary cannot force the protocol into selecting a puzzle for which they have already precomputed this trapdoor.

Lemma 1 (Epoch Key Unpredictability). *Let $\mathbb{G} = \langle g \rangle$ be a cyclic group of prime order q in which the discrete logarithm problem is hard. Let $H : \{0, 1\}^* \rightarrow \mathbb{G}$ be a hash-to-curve function modeled as a random oracle into the prime-order*

subgroup. For epoch $i + 1$, the protocol derives the next encryption key as:

$$ek_{i+1} := H(ek_i \parallel n_i)$$

where n_i is the L1 block hash at the moment of the previous epoch’s resolution. Consider an adversary $\mathcal{A}$ that:

1. *Precomputes a set $S = \{g^{x_1}, \dots, g^{x_M}\} \subseteq \mathbb{G}$ of size M for which it knows the discrete logs $x_j \in \mathbb{Z}_q$.*
2. *Can grind up to K distinct block hashes per epoch (e.g., by withholding headers or manipulating L1 transaction ordering) before n_i is finalized.*

Then for any epoch i , the probability that the adversary forces the protocol into a precomputed key is bounded by:

$$\Pr[ek_{i+1} \in S] \leq \frac{MK}{q}$$

Across T independent epochs, the success probability is at most $\min\{1, MKT/q\}$ by a union bound.

Proof. Modeling H as a random oracle into $\mathbb{G}$ implies that each unique query returns an indistinguishable, uniformly distributed element in $\mathbb{G}$. For any fixed element $Y \in \mathbb{G}$, $\Pr[H(x) = Y] = 1/q$. Hence, for a fixed precomputed set S with $|S| = M$, $\Pr[H(x) \in S] = M/q$.

If the L1 proposer $\mathcal{A}$ evaluates $H(ek_i \parallel n_i)$ on up to K distinct, valid block hashes $n_{i,1}, \dots, n_{i,K}$ and then finalizes the one that maximizes their advantage, a union bound yields:

$$\Pr[\exists j : H(ek_i \parallel n_{i,j}) \in S] \leq \sum_{j=1}^K \Pr[H(\cdot) \in S] = K \cdot \frac{M}{q}$$

Repeating this grinding attack across T epochs gives at most MKT/q success probability by a standard union bound over the epochs. $\square$

Remark 5. To target $2^{-\lambda}$ security against this specific pre-computation attack over T epochs, parameters must satisfy $q \geq 2^\lambda \cdot MKT$. For standard security (e.g., $\lambda = 128$) and $q \approx 2^{256}$, the bound remains cryptographically negligible ($\leq 2^{-128}$) even if an adversary commands enormous computational precomputation resources ($MKT \leq 2^{128}$). Thus, the only viable vector to recover dk_i is via real-time computation *after* ek_i is revealed.

Lemma 2 (Pre-execution Privacy). *Assume the continuous discrete logarithm search is modeled as a memoryless Poisson process. Let T be the protocol’s calibrated expected time to solve the active puzzle I_{ek_i} , and let L be the strict L1 cutoff time (the commit window) for epoch i . The probability that any adversary, possessing computational power proportional to the difficulty target, successfully recovers the decryption key dk_i before the commit window L closes is bounded by:*

$$\Pr[\text{Premature Decryption}] \leq 1 - e^{-L/T}$$

Proof. By Remark 2, after Lemma 1 rules out precomputed favorable keys except with negligible probability, trapdoor recovery during the commit window is modeled as aggregate real-time DLP search under the PMS abstraction. Thus, the first successful recovery event is treated as a Poisson process with calibrated expected time T .

Let the continuous random variable X denote the time elapsed until the puzzle is solved. Given the protocol's calibrated expected time T , the rate parameter of the associated exponential distribution is $\lambda = 1/T$. The probability of an adversary finding the solution at any time $t \leq L$ is exactly the cumulative distribution function (CDF) of this exponential distribution:

$$\Pr[X \leq L] = \int_0^L \frac{1}{T} e^{-t/T} dt = 1 - e^{-L/T}$$

Therefore, assuming the adversary cannot bypass the underlying discrete logarithm hardness via precomputation (as bounded by Lemma 1), the maximum probability of a privacy breach occurring before the L1 commit phase concludes is $1 - e^{-L/T}$. $\square$

Lemma 3 (Guaranteed Decryption). *Let T^* be the protocol's calibrated expected solution time under a baseline network computational power C^* . Assume there exists an available honest computational power $C_{\text{honest}} = \alpha C^*$ continuously attempting to solve the active decryption puzzle, where $\alpha \in (0, 1]$.*

For any maximum acceptable delay threshold $t_{\max}$ and negligible failure probability ϵ , if the honest computational fraction satisfies:

$$\alpha \geq \frac{T^*}{t_{\max}} \ln\left(\frac{1}{\epsilon}\right)$$

then, DecryptChain satisfies (eventual) Guaranteed Decryption: every committed encrypted transaction is successfully decrypted within time $t_{\max}$ with probability at least $1 - \epsilon$.

Proof. Let tx be an encrypted transaction committed during an epoch with an active puzzle I_{ek_i} . Due to the injectivity of DecryptChain puzzles, a unique decryption key dk_i exists. Given that the puzzle difficulty was calibrated to yield an expected time T^* under power C^* , the actual expected time for the honest portion to find the solution is inversely proportional to their active computation power: $T_{\text{honest}} = T^*/\alpha$.

The continuous parallelized search for the discrete logarithm (e.g., via Pollard's rho) implies a memoryless Poisson process. Let the random variable X denote the time elapsed until the honest network recovers dk_i . The probability of successful decryption within the maximum delay $t_{\max}$ is given by the cumulative distribution function of the exponential distribution with rate $\lambda = 1/T_{\text{honest}} = \alpha/T^*$:

$$\Pr[X \leq t_{\max}] = 1 - e^{-\alpha t_{\max}/T^*}$$

To guarantee liveness with overwhelming probability, we require the failure probability to be bounded by ϵ :

$$1 - e^{-\alpha t_{\max}/T^*} \geq 1 - \epsilon \implies e^{-\alpha t_{\max}/T^*} \leq \epsilon$$

Taking the natural logarithm yields:

$$-\frac{\alpha \cdot t_{\max}}{T^*} \leq \ln(\epsilon) \implies \alpha \geq \frac{T^*}{t_{\max}} \ln\left(\frac{1}{\epsilon}\right)$$

Thus, provided the honest computational power meets this lower bound, the honest participants will find the valid solution within time $t_{\max}$ with probability $\geq 1 - \epsilon$. $\square$

Theorem 5 (DecryptChain satisfies Recoverability). *Assume the conditions of Lemma 1: the hash-to-curve function is modeled as a random oracle into the prime-order subgroup, and the protocol parameters make the precomputation and grinding probability negligible. Then DecryptChain satisfies Recoverability as in Definition 2.*

Proof. Fix a target epoch e . The public artifact A_e consists of the active encryption key ek_e together with the public group parameters, and the trapdoor τ_e is the corresponding decryption key dk_e satisfying $g^{\text{dk}_e} = \text{ek}_e$. By the injectivity of DecryptChain puzzles, this decryption key is unique.

It remains to show that state generated before epoch e does not give a reusable shortcut for recovering dk_e within epoch e 's commit window. The protocol never samples dk_e , distributes shares of dk_e , or stores a long-lived opening capability from which dk_e can be derived. Instead, ek_e is obtained through the hash-to-curve step from public protocol state. By Lemma 1, an adversary that carries arbitrary state from earlier epochs can force ek_e into a set for which it already knows the discrete logarithm only with negligible probability under the stated parameters.

Conditioned on the complement of this negligible event, the past state $\text{state}_{<e}$ may contain previous trapdoors, miner state, or any other internal state generated before epoch e , but it does not contain opening information for the fresh epoch- e trapdoor. Therefore, giving $\text{state}_{<e}$ to a PPT extractor can only change its probability of recovering dk_e within epoch e 's commit window by a negligible amount. The remaining way to recover dk_e is the ordinary real-time attack on the epoch- e discrete-logarithm instance, whose premature success probability is bounded in Lemma 2. Hence DecryptChain satisfies Recoverability. $\square$

B Succinct Verification that $\#E(\mathbb{F}_p) = n$

In our target applications, the *difficulty* (and thus security/cost) is tuned by the hardness of the discrete logarithm on an elliptic curve. To control the difficulty of breaking the discrete log, we periodically switch to a fresh curve whose *prime* order

lands in a prescribed range. Computing such a curve *on-chain* is prohibitive; instead, we generate it off-chain and attach a succinct ZK certificate that $\#E(\mathbb{F}_p) = n$ with n prime. The smart contract verifies this single proof and then checks that n satisfies the difficulty policy; it accepts the curve only if both conditions hold. We first start by laying intuition, then move to the Formal description.

Our goal is to have a short zero-knowledge certificate that convinces a verifier that the elliptic-curve group over $\mathbb{F}_p$ has *exactly* n points.

Why not enumerate? For any elliptic curve over a field with p elements, the point count is very close to $p+1$. Hasse's bound [20] pins the group size to the interval

$$\#E(\mathbb{F}_p) \in [p+1-2\sqrt{p}, p+1+2\sqrt{p}] \quad (\text{width } 4\sqrt{p}).$$

When p is 256-bit, that "window" already contains astronomically many integers (on the order of 2^{128}), so scanning candidates or counting on-chain is infeasible.

Why primality tests? We first test that p and n are prime (fast Miller–Rabin) [26, 32]. If p were composite, we would not be in a true finite field and curve arithmetic could break. Crucially for security, the discrete-log problem on $E(\mathbb{F}_p)$ is hard only when the group has a large *prime* order: if $\#E = k \cdot n$ or n itself is composite, the Pohlig–Hellman reduction solves DLP in time roughly $O(\sqrt{q_{\max}})$ where $q_{\max}$ is the largest prime dividing $\#E$ [29]. Requiring n to be prime and then certifying *exactly* $\#E = n$ (so the cofactor $k = 1$) rules out small-subgroup vulnerabilities and ensures the intended prime-order group for cryptographic use.

How do we test that n divides the group size? In any finite group of size m , applying the group operation m times yields the identity (the analogue of " $a^m = 1$ " in multiplicative groups). Thus, if $\#E(\mathbb{F}_p) = n$, then for every point P we have $[n]P = O$. If instead $\#E(\mathbb{F}_p) = k \cdot n$ with cofactor $k \geq 2$, then $[n]P$ lies in a leftover subgroup of size k , and a *random* point hits the identity only with probability $1/k$.

Sample s independent points $P_1, \dots, P_s$ (e.g., via hash-to-curve [16]) and check $[n]P_i = O$ for each i , paying s scalar multiplications by n . If $\#E(\mathbb{F}_p) = k \cdot n$ with $k \geq 2$, then each check succeeds with probability $1/k$, and independence yields total false-accept probability $(1/k)^s$.

Remark 6. One might try to *compress* the s checks into a single one by sampling coefficients $c_i \in \{1, \dots, n-1\}$ and testing $[n]Q = O$ for $Q = \sum_{i=1}^s [c_i]P_i$. Although correctness holds by linearity of $[n]$, the soundness collapses: in the bad case, writing $Q_i := [n]P_i$ and $H := \text{Im}([n])$ (a subgroup of size k), we have $[n]Q = \sum_i [c_i]Q_i$ in H . On H , scalar actions depend only on $c_i \bmod k$; with random c_i these residues are (nearly) uniform in $\mathbb{Z}_k$, and for random P_i the Q_i generate H with overwhelming probability. Hence $\sum_i [c_i]Q_i$ is uniform in H and equals O with probability exactly $1/k$, *independent* of s . Intuitively, one random linear equation in a k -element

abelian group removes only one degree of freedom. Therefore, a single random linear combination does not retain the $(1/k)^s$ amplification and should not be used in place of the independent checks.

Formal description. We now give a single, non-interactive relation that certifies $\#E(\mathbb{F}_p) = n$ for a short Weierstrass curve $E/\mathbb{F}_p : y^2 = x^3 + Ax + B \bmod p$. The certificate bundles curve sanity, primality of p and n , Hasse inclusion, and a cofactor-exclusion test into one succinct ZK proof.

Inputs and notation. Public instance $\text{inst} = (p, A, B, n)$ with $p \geq 5$, $A, B \in \mathbb{Z}_p$, $n \in \mathbb{N}$. Let $\Delta = 4A^3 + 27B^2 \bmod p$, and let O denote the identity. Let $\text{H2C} : \{0, 1\}^* \rightarrow E(\mathbb{F}_p)$ be a deterministic, indifferentiable hash-to-curve map, and let H be a SNARK-friendly hash (e.g., Poseidon2). Concatenation is denoted by $\| \cdot \|$, and dom is a fixed domain tag.

Single-parameter soundness. Fix a target soundness $2^{-\lambda}$ (e.g., $\lambda = 128$). From λ derive:

$$r \leftarrow \left\lceil \frac{\lambda+2}{2} \right\rceil \quad (\text{Miller–Rabin rounds for each of } p \text{ and } n), \quad (3)$$

$$m \leftarrow \lambda + 1 \quad (\text{number of randomized points}). \quad (4)$$

With these choices, the total (union-bound) soundness error is at most $2^{-\lambda}$; see Theorem 6.

Unified certificate relation $\mathcal{R}$. A witness w consists of the data needed to verify r strong Miller–Rabin (SMR) rounds for p and for n (i.e., bases and intermediate squares). The relation $\mathcal{R}(\text{inst}, w) = 1$ holds iff:

1. **Curve sanity.** $\Delta \not\equiv 0 \pmod{p}$.
2. **Primality of p and n .** The SMR test with r rounds accepts p and n .
3. **Hasse inclusion.** $|n - (p+1)| \leq 2\sqrt{p}$.
4. **Cofactor exclusion via s independent checks.** For each $j = 1, \dots, s$, derive an independent point

$$P_j \leftarrow \text{H2C}(\text{dom} \| p \| A \| B \| n \| j),$$

and enforce the kernel equality

$$[n]P_j = O.$$

Remark 7. Under the random-oracle model and when inst is fixed independently of the oracle, no external beacon is required. In settings where the prover can *choose* inst adaptively to the oracle, one should domain-separate $H/\text{H2C}$ by an additional public seed to preclude grinding (e.g., previous block hash).

B.1 Correctness and Soundness

Lemma 4 (Completeness). *If p and n are prime, $\Delta \not\equiv 0 \pmod{p}$, and $\#E(\mathbb{F}_p) = n$, then $\mathcal{R}(\text{inst}, w) = 1$ for any $\lambda \geq 1$.*

Proof Sketch. SMR accepts primes with probability 1. Hasse inclusion holds since $n = \#E$. For any P^* , $[n]P^* = O$ when $\#E = n$. $\square$

Theorem 6 (Soundness). *Model H and H2C as random oracles. Let $\#E(\mathbb{F}_p) = k \cdot n$ with $k \geq 1$. With $r = \lceil (\lambda + 2)/2 \rceil$ and $m = \lambda + 1$, the verifier accepts with probability at most $2^{-\lambda}$.*

Proof Sketch. A strong MR round errs with probability $\leq \frac{1}{4}$ on any odd composite. Testing both p and n with r rounds gives MR error $\leq 2 \cdot 4^{-r} = 2^{-(2r-1)} \leq 2^{-(\lambda+1)}$. Conditioned on SMR success and Hasse inclusion, each P_j is close to uniform in $E(\mathbb{F}_p)$; hence $[n]P_j$ is uniform in the size- k image subgroup H , and equals O with probability $1/k \leq 1/2$ per round. Across s independent rounds this contributes at most $k^{-s} \leq 2^{-s} \leq 2^{-(\lambda+1)}$. $\square$

B.2 Complexity and Parameterization

For target $2^{-\lambda}$ soundness we set $r = \lceil (\lambda + 2)/2 \rceil$ (Miller–Rabin rounds for p and n) and $s = \lambda + 1$ (independent cofactor checks). The prover’s constraint cost is dominated by:

- s scalar multiplications by the public scalar n (one per check $[n]P_j = O$), plus the associated group additions;
- $2r = \lambda + 2$ strong Miller–Rabin rounds across (p, n) , each a modular exponentiation of size $\log p$ or $\log n$;
- $O(s)$ evaluations of H2C and the SNARK-friendly hash H for domain-separated sampling of the P_j .

All checks are enforced in a single circuit, yielding one succinct proof and a constant-time verifier (the SNARK verifier), independent of p and n .

Parameter shortcut. If n satisfies Hasse’s inclusion $|n - (p+1)| \leq 2\sqrt{p}$ and $n > 4\sqrt{p}$, then the Hasse interval $[p+1 - 2\sqrt{p}, p+1 + 2\sqrt{p}]$ of width $4\sqrt{p}$ can contain at most one multiple of n . Consequently, once we establish divisibility $n \mid \#E(\mathbb{F}_p)$, it follows that $\#E(\mathbb{F}_p) = n$ (i.e., the cofactor $k = 1$). In this regime, we may set $s = 1$ (one kernel check $[n]P = O$); the only remaining error is from that single probabilistic kernel check and the primality tests.

C Reference Code: Finding a Curve of Prescribed Order

Implementation. We implement the Bröker–Stevenhagen CM algorithm for constructing elliptic curves of a prescribed

prime order n [6]. Our Sage code follows the paper’s steps (Cornacchia: $p = n + 1 \pm x$, reduction of $H_{-D}(X) \pmod{p}$, and a constant-time quadratic-twist check).

```
from itertools import combinations

def cornacchia(d, m):
    assert (gcd(d, m) == 1)
    try:
        rs = [ ZZ(x) for x in
            ↪ Zmod(m) (-d).sqrt(all=True) ]
    except:
        return None
    for r in rs:
        r0 = m; r1 = r
        while r0^2 >= m:
            r0, r1 = r1, r0 % r1
            if is_square((m - r0^2)/d):
                return r0, sqrt((m - r0^2)/d)
    return None

def find_curve_of_order(n):
    r = 0
    logn = int(n.log(2).n())
    while True:
        p = [ legendre_symbol(-1, p)*p for p in
            ↪ prime_range(max(3, r*logn), (r+1)*logn)
            ↪ if legendre_symbol(n, p) == 1 ]
        S = [ p for p in p ]
        for i in range(1, len(p)):
            for L in combinations(S, i):
                D = prod(L)
                if D % 8 != 5:
                    continue
                solution = cornacchia(-D, 4*n)
                if solution is None:
                    continue
                (x, y) = solution
                if is_prime(n+1+x):
                    p = n+1+x
                elif is_prime(n+1-x):
                    p = n+1-x
                else:
                    continue
                P = hilbert_class_polynomial(D)
                roots = P.roots(ring=GF(p))
                if len(roots) > 0:
                    E = EllipticCurve_from_j(roots[ ]
                        ↪ 0][0]).change_ring(GF(p))
                    if E.order() == n:
                        return E
                else:
                    return E.quadratic_twist()

    r += 1

primes_near_2_127 = [
    170141183460469231731687303715884105727,
    170141183460469231731687303715884105703,
```

```

170141183460469231731687303715884105689,
170141183460469231731687303715884105757,
170141183460469231731687303715884105773,
170141183460469231731687303715884105793
]
primes_near_2_256 = [
11579208923731619542357098500868790785326998466,
→ 5640564039457584007913129639747,
11579208923731619542357098500868790785326998466,
→ 5640564039457584007913129639579,
11579208923731619542357098500868790785326998466,
→ 5640564039457584007913129639501,
11579208923731619542357098500868790785326998466,
→ 5640564039457584007913129640233,
11579208923731619542357098500868790785326998466,
→ 5640564039457584007913129640237,
11579208923731619542357098500868790785326998466,
→ 5640564039457584007913129640293
]
for i in (primes_near_2_127 + primes_near_2_256):
    print(find_curve_of_order(i))

```

Complexity. Under standard heuristics (small D , fast square roots mod m , fast class polynomials mod p), the expected running time is $\tilde{O}((\log n)^c)$ for a small constant c ; in practice, it completes in milliseconds to seconds on commodity hardware.

We implemented the miner-side curve finder (Appendix C) in SageMath and ran it on a commodity desktop (AMD Ryzen 7 3700X). For twelve target prime orders (six near 2^{127} and six near 2^{256}), the end-to-end wall-clock time for a single run of `ecgen.sage` was **2m03.6s** (real), with `user=2m02.9s` and `sys=0.63s`; this is about **10.3s per curve** on average. This confirms that curve selection is fast enough for miners to perform locally at retarget time.

Sample output. Below are the curves produced in that run (one per target n):

```

Elliptic Curve defined by  $y^2 = x^3 +$ 
→  $29825378467770542984135446927558639468x +$ 
→  $54083907264064377833823915102218955679$  over
→ Finite Field of size
→  $170141183460469231709232497652167941199$ 
Elliptic Curve defined by  $y^2 = x^3 +$ 
→  $166704574998777851332135925087901176418x +$ 
→  $2501792355787950223526286414255312734$  over
→ Finite Field of size
→  $170141183460469231735139573472496610761$ 
Elliptic Curve defined by  $y^2 = x^3 +$ 
→  $5835705960842305188851787587145231939x +$ 
→  $98653831568404699155441393320732712297$  over
→ Finite Field of size
→  $170141183460469231707179993871836100599$ 

```

```

Elliptic Curve defined by  $y^2 = x^3 +$ 
→  $1434089606890065762630983049280510878x +$ 
→  $103284139592733586440116893949965403185$  over
→ Finite Field of size
→  $170141183460469231740939206871127759249$ 
Elliptic Curve defined by  $y^2 = x^3 +$ 
→  $57985085020697206343581501399891348248x +$ 
→  $65674023084750787642239564949861974476$  over
→ Finite Field of size
→  $170141183460469231719169500028683362243$ 
Elliptic Curve defined by  $y^2 = x^3 +$ 
→  $74303471570358586521769780306807836411x +$ 
→  $142631150465124477830319751487984594805$  over
→ Finite Field of size
→  $170141183460469231754403863118328463863$ 
Elliptic Curve defined by  $y^2 = x^3 +$ 
→  $24963013499087561931964830351747555344721058543,
→ 310859651390459937971963486428x +$ 
→  $69987288996194560123614897879958660149968027795,$ 
→  $087187875826915983502129576939$  over Finite Field
→ of size  $115792089237316195423570985008687907853,$ 
→  $801471696372379092604433305066114081027$ 
Elliptic Curve defined by  $y^2 = x^3 +$ 
→  $10491309742481062958589500528506609975990619488,$ 
→  $7592880275981291737031591012624x +$ 
→  $34400763528950338162925379285181863571400166929,$ 
→  $344586176077580773844707561982$  over Finite Field
→ of size  $115792089237316195423570985008687907853,$ 
→  $634669878699707804857089810265963587517$ 
Elliptic Curve defined by  $y^2 = x^3 +$ 
→  $10020762455541125099711312300866887719462816255,$ 
→  $8686399732525417315202133466991x +$ 
→  $16583568605012532250952996016611633811123902143,$ 
→  $75485457510422238929307589272$  over Finite Field
→ of size  $115792089237316195423570985008687907853,$ 
→  $831873791641224224434627315647913457561$ 
Elliptic Curve defined by  $y^2 = x^3 +$ 
→  $95198743706638659015924324373110976406448058556,$ 
→  $735217921521667853979578154778x +$ 
→  $63182328071332379106337558054647811775947488510,$ 
→  $307783526324926708722137575777$  over Finite Field
→ of size  $115792089237316195423570985008687907852,$ 
→  $942313707343596112675143906678738492773$ 
Elliptic Curve defined by  $y^2 = x^3 +$ 
→  $40294862990429573945522434997582079334351644576,$ 
→  $526923228849293002070585590025x +$ 
→  $49568664143274310184022750283738288121778358584,$ 
→  $37820294846034096652214086348$  over Finite Field
→ of size  $115792089237316195423570985008687907853,$ 
→  $362405928232336521054202364603508119867$ 
Elliptic Curve defined by  $y^2 = x^3 +$ 
→  $21624043899848771590996981939426634926865192018,$ 
→  $115608639724267336766285356327x +$ 
→  $36160615241262567997003876289922091910656001037,$ 
→  $943386874484423811782285071160$  over Finite Field
→ of size  $115792089237316195423570985008687907852,$ 
→  $968823485740947407128351607970905561469$ 

```

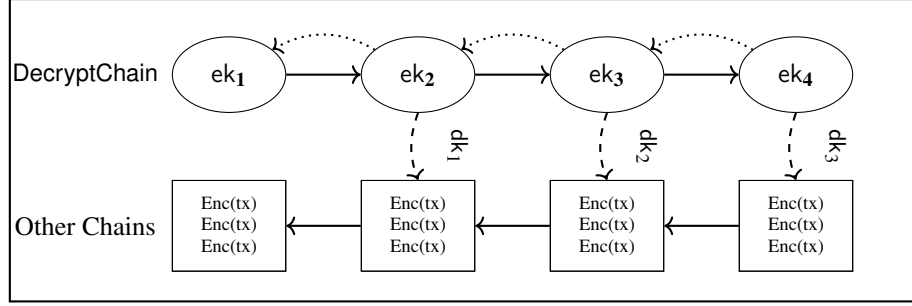


Figure 3: A high-level illustration of the universal DecryptChain and a base chain. Note that here the decryption key dk_i is publicly released so that any other chain (and not just the base chain) can use it to decrypt and execute their transactions.

D DecryptChain: A Universal Variant

Our main DecryptChain construction (see Section 4) works as an L2 protocol that uses its underlying blockchain (*e.g.*, Ethereum) for block commitment (*i.e.*, consensus) and data availability. However, we can extend the functionality of our system to serve more than one chain in a universal manner. That is, any chain interested in deploying a permissionless encrypted mempool can benefit from our DecryptChain simultaneously. We can do this extension with slight modification to our main design. First, the miner i on DecryptChain should explicitly reveal the decryption key dk_i so that all the other chains have access to the key for decrypting their own transactions. Given that the corresponding encryption key is already known and published by the DecryptChain contract, anyone can check the validity of the (unique) decryption key. Recall that in our original design, the miner does not leak the decryption key on-chain and just provides a validity proof for the state update on the underlying L1. Second, the winning miner needs to go through an interactive commit-reveal process binding its claim to the committed solution and its payout address to protect their work reward from being stolen by others. A pictorial presentation of our universal variant is depicted in 3.

On the limitation of a commit-reveal style solution. While using a commit-reveal style solution can plausibly solve the problem of reward theft, it comes with limitations in one way or another. Next, we briefly discuss potential pitfalls:

Liveness violation. A liveness issue might happen in case the miner decides not to publish the decryption key after being determined as the eligible miner (*i.e.*, in the reveal phase). This might be due to bribing, where the leakage of the decryption key allows executing some transaction that is not of interest to the briber. Punishing the silent miner and letting others come forward still suffers from the same issue as the same attacker might carry out a sibyl attack and try with different addresses.

Spamming the network. The commit phase should be verifiable so that any invalid solution gets rejected. Other-

wise, an attacker can spam the network by sending many invalid commitments (*i.e.*, without knowing the decryption key). Limiting the commit window time to protect from spamming may cause a liveness issue, as no one might have found the key until then.

Delaying the winner. After the commit phase, the attacker can potentially delay the winner (*e.g.*, via DDOS) in the hope of learning the solution in the reveal phase and do a commit-reveal on its own to get the reward. This scenario might be hard to realize in practice, however, due to the censorship resistance property of the underlying L1 blockchains and the need for precise timing analysis.